

Implementação do Algoritmo Rapidly-exploring Random Tree (RRT) para Planejamento de Caminho em Robótica Móvel

Mariana Lins

Universidade Federal de Alagoas (UFAL)

Projeto Novo Ensino Suplementar (NES)

Disciplina de Robótica Móvel

Maceió, Alagoas, Brasil

Abstract—O planejamento de caminho é uma das tarefas fundamentais da robótica móvel, permitindo que robôs naveguem de forma autônoma entre uma posição inicial e um objetivo evitando obstáculos. Neste trabalho foi implementado o algoritmo Rapidly-exploring Random Tree (RRT) utilizando Python no ambiente RoboticsAcademy (JdeRobot). A solução desenvolvida realiza a construção incremental da árvore de exploração, verificação de colisões, reconstrução do caminho e navegação do robô até o destino. Os resultados demonstram o funcionamento do algoritmo por meio da geração da árvore de busca, do caminho encontrado e da execução da navegação.

Index Terms—Robótica Móvel, Planejamento de Caminho, Rapidly-exploring Random Tree, RRT, RoboticsAcademy, Python.

I. INTRODUÇÃO

A robótica móvel tem se destacado como uma área de grande relevância tecnológica, com aplicações que vão desde veículos autônomos até robôs de exploração em ambientes hostis. Um dos problemas centrais nessa área é o planejamento de caminho, que consiste em determinar uma trajetória viável para um robô se deslocar de um ponto inicial a um ponto final, desviando de obstáculos presentes no ambiente [1].

Diversos algoritmos foram propostos para resolver esse problema, sendo os métodos baseados em amostragem uma classe particularmente eficiente para espaços contínuos e de alta dimensionalidade. Entre eles, o algoritmo *Rapidly-exploring Random Tree* (RRT), proposto por LaValle em 1998 [1], destaca-se por sua simplicidade, eficiência e capacidade de explorar rapidamente o espaço de configurações.

O RRT é um algoritmo incremental que constrói uma árvore de caminhos a partir da posição inicial, expandindo-a aleatoriamente em direção a pontos amostrados no espaço. A cada iteração, um ponto aleatório é gerado, o nó mais próximo da árvore é selecionado e um novo nó é criado na direção desse ponto, respeitando restrições de distância máxima. O processo continua até que a árvore atinja a região do objetivo ou um número máximo de iterações seja atingido.

Neste trabalho, o algoritmo RRT foi implementado em Python e integrado ao ambiente de simulação RoboticsAcademy (JdeRobot) [2]. O objetivo principal é validar o funcionamento do RRT em um cenário de robótica móvel,

demonstrando sua capacidade de gerar caminhos viáveis em ambientes com obstáculos estáticos.

A implementação desenvolvida é composta por quatro módulos principais: (i) geração incremental da árvore de exploração, (ii) verificação de colisões com obstáculos, (iii) reconstrução do caminho a partir da árvore, e (iv) navegação do robô até o destino. Os resultados obtidos são apresentados por meio de visualizações gráficas da árvore gerada e do caminho encontrado, além da execução da navegação.

Este artigo está organizado da seguinte forma: a Seção II apresenta os fundamentos teóricos do algoritmo RRT. A Seção III descreve a metodologia adotada, incluindo o ambiente de simulação e as etapas de implementação. A Seção IV detalha a implementação em Python com trechos do código real. A Seção V apresenta e discute os resultados obtidos. Finalmente, a Seção VI traz as conclusões e propostas para trabalhos futuros.

II. FUNDAMENTAÇÃO TEÓRICA

A. Planejamento de Caminho em Robótica

O planejamento de caminho é um problema clássico da robótica que consiste em encontrar uma sequência de configurações válidas que conectem um estado inicial a um estado final, respeitando as restrições cinemáticas e dinâmicas do robô, além de evitar colisões com obstáculos no ambiente [1]. Formalmente, dado o espaço de configurações $\mathcal{C}$, o espaço livre $\mathcal{C}_{free}$ (configurações sem colisões), uma configuração inicial $q_{init} \in \mathcal{C}_{free}$ e uma configuração final $q_{goal} \in \mathcal{C}_{free}$, o problema é encontrar uma função contínua $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$ tal que $\tau(0) = q_{init}$ e $\tau(1) = q_{goal}$.

Os métodos de planejamento podem ser classificados em diversas categorias: métodos baseados em grafos (como o A* e o Dijkstra), métodos baseados em campos potenciais, métodos de decomposição celular e métodos baseados em amostragem [1]. Esses últimos são particularmente adequados para espaços de alta dimensionalidade, onde a construção explícita do espaço de configurações é inviável computacionalmente.

B. O Algoritmo RRT

O algoritmo RRT foi proposto por LaValle [1] e se tornou uma das técnicas mais populares para planejamento de caminho em espaços contínuos. Sua principal característica é a construção incremental de uma árvore de exploração que tende a "crescer" em direção às regiões não exploradas do espaço, o que o torna eficiente para ambientes grandes e complexos.

O funcionamento do RRT é descrito pelo pseudocódigo a seguir:

```
Entrada:  $q_{init}, q_{goal}, \mathcal{C}_{free}, \Delta t, K_{max}$   
 $T \leftarrow$  Inicializar árvore com  $q_{init}$   
for  $k = 1$  até  $K_{max}$  do  
   $q_{rand} \leftarrow$  Amostrar um ponto aleatório em  $\mathcal{C}$   
   $q_{near} \leftarrow$  Encontrar o nó mais próximo de  $q_{rand}$  na árvore  
   $q_{new} \leftarrow$  Estender de  $q_{near}$  na direção de  $q_{rand}$  com passo  $\Delta t$   
  if  $q_{new} \in \mathcal{C}_{free}$  e a aresta  $q_{near} \rightarrow q_{new}$  é livre then  
     $T \leftarrow T \cup \{q_{new}\}$   
    Conectar  $q_{new}$  a  $q_{near}$  na árvore  
    if  $\|q_{new} - q_{goal}\| < \epsilon$  then  
      retorna Caminho reconstruído de  $q_{goal}$  até  $q_{init}$   
    end if  
  end if  
end for  
retorna Falha
```

A eficiência do RRT está na sua capacidade de explorar o espaço de forma balanceada: a aleatoriedade evita que a árvore fique presa em mínimos locais, enquanto a tendência de crescimento em direção a regiões não exploradas acelera a convergência para o objetivo. Entretanto, o RRT não garante encontrar uma solução ótima, apenas uma solução viável [1].

C. Variações e Melhorias do RRT

Diversas variações do RRT foram propostas para melhorar seu desempenho e qualidade das soluções. O RRT-Connect [4] utiliza duas árvores crescendo simultaneamente a partir da origem e do destino, aumentando a eficiência. O RRT* (RRT-Star) [5] introduz a rewiring da árvore, garantindo convergência assintótica para o caminho ótimo. O Informed-RRT* [6] restringe a amostragem a uma elipse definida pelo caminho encontrado, melhorando a convergência para a solução ótima.

Embora existam essas variantes, a implementação básica do RRT foi adotada neste trabalho por sua simplicidade e adequação ao ambiente de simulação proposto.

III. METODOLOGIA

A. Ambiente de Simulação: RoboticsAcademy

O RoboticsAcademy [2] é uma plataforma educacional desenvolvida pelo JdeRobot que oferece um conjunto de exercícios práticos para aprendizado de robótica. A plataforma é baseada no simulador Gazebo e fornece interfaces em Python para controle de robôs em diversos cenários.

No exercício de planejamento de caminho, o robô deve navegar em um ambiente com obstáculos estáticos desde uma posição inicial até um destino definido. O ambiente fornece:

- Coordenadas da posição inicial e do objetivo;

- Mapa do ambiente com informações sobre obstáculos;
- Uma API para controle do robô (movimento linear e angular);
- Funções para verificação de colisões.

A integração entre o Python e o RoboticsAcademy foi realizada por meio da biblioteca `gym-jderobot` e do módulo `HAL` para acesso direto aos sensores e atuadores.

B. Escolha do RRT e Motivação

O algoritmo RRT foi escolhido por sua simplicidade de implementação e por ser amplamente utilizado em aplicações de robótica móvel. Sua natureza baseada em amostragem aleatória torna-o robusto para ambientes com obstáculos complexos, sendo uma excelente opção para o escopo deste trabalho.

Além disso, o RRT não requer um modelo explícito do espaço de configurações, apenas uma função que verifica se uma determinada configuração é válida (sem colisões). Isso facilita sua integração com o simulador, que já fornece a função de verificação de colisão.

C. Etapas de Desenvolvimento

O desenvolvimento foi dividido em quatro etapas principais:

- 1) **Configuração do ambiente:** Instalação do RoboticsAcademy e familiarização com a API de controle do robô e verificação de colisões.
- 2) **Implementação do RRT:** Codificação do algoritmo conforme o pseudocódigo apresentado, incluindo a estrutura de dados para a árvore (lista de nós com coordenadas e referência ao pai), função de distância euclidiana e função de extensão.
- 3) **Verificação de colisões:** Utilização da função de verificação de colisão do simulador para testar se um nó ou uma aresta está livre de obstáculos.
- 4) **Reconstrução do caminho e navegação:** Após a árvore alcançar o objetivo, o caminho é reconstruído percorrendo os nós pais desde o objetivo até a raiz. Em seguida, o robô é controlado para seguir o caminho encontrado.

D. Parâmetros do RRT

Os parâmetros adotados na implementação, conforme o código real, foram:

- Distância máxima de extensão (Δt): 18 pixels (unidades do mapa);
- Probabilidade de direcionar a amostra para o objetivo: 15% (`goal_bias=0.15`);
- Número máximo de iterações (K_{max}): 5000;
- Tolerância para alcançar o objetivo: definida como $1.5 \times \Delta t$ no código.

Esses valores foram definidos empiricamente durante os testes iniciais, buscando um equilíbrio entre tempo de execução e qualidade da solução, considerando a resolução do mapa fornecido.

IV. IMPLEMENTAÇÃO

A implementação foi realizada em Python 3, utilizando as bibliotecas `numpy`, `opencv-python` para manipulação de imagens e `math` para cálculos geométricos. A seguir, são apresentados os principais trechos do código real, extraídos do arquivo `academy.py`.

A. Estrutura da Árvore e Classe do Planejador

A árvore é armazenada como uma lista de nós, onde cada nó é representado por uma tupla nomeada (`Node`) contendo as coordenadas (x, y) e o índice do nó pai.

```
from collections import namedtuple
Node = namedtuple('Node', ['x', 'y', 'parent'])

class RRTPlanner:
    def __init__(self, map_array, step_size=15, goal_bias=0.2, max_iter=4000):
        self.map = map_array
        self.step_size = step_size
        self.goal_bias = goal_bias
        self.max_iter = max_iter
        self.tree = []
        self.path = []
```

B. Amostragem Aleatória e Tendência ao Objetivo

A função `sample_random_point` gera um ponto aleatório dentro dos limites do mapa, com probabilidade `goal_bias` de retornar o próprio objetivo, acelerando a convergência.

```
def sample_random_point(self, goal):
    if random.random() < self.goal_bias:
        return goal
    h, w = self.map.shape[:2]
    return (random.randint(0, w-1), random.randint(0, h-1))
```

C. Encontrar o Nó Mais Próximo

A distância euclidiana é utilizada para encontrar o nó mais próximo do ponto amostrado.

```
def find_nearest_node(self, point):
    nearest = self.tree[0]
    min_dist = self.distance(nearest, point)
    for node in self.tree:
        d = self.distance(node, point)
        if d < min_dist:
            min_dist = d
            nearest = node
    return nearest
```

D. Extensão da Árvore

A função `steer` calcula o novo ponto na direção do ponto amostrado, respeitando o tamanho do passo.

```
def steer(self, from_node, to_point):
    dx = to_point[0] - from_node.x
    dy = to_point[1] - from_node.y
    dist = math.hypot(dx, dy)
    if dist <= self.step_size:
        return to_point
    scale = self.step_size / dist
    return (int(from_node.x + dx * scale), int(from_node.y + dy * scale))
```

E. Verificação de Colisão

A função `is_collision_free` percorre o segmento de reta entre dois pontos, verificando se algum pixel corresponde a um obstáculo no mapa de ocupação.

```
def is_collision_free(self, from_node, to_point):
    steps = int(max(abs(to_point[0] - from_node.x),
                    abs(to_point[1] - from_node.y)))
```

```
if steps == 0:
    return True
for i in range(steps + 1):
    t = i / steps
    x = int(from_node.x + t * (to_point[0] - from_node.x))
    y = int(from_node.y + t * (to_point[1] - from_node.y))
    if x < 0 or y < 0 or y >= self.map.shape[0] or x >= self.map.shape[1]:
        return False
    if self.map[y, x] > 0.5: # obstáculo
        return False
return True
```

F. Reconstrução do Caminho

Ao encontrar um nó dentro da tolerância do objetivo, o caminho é reconstruído percorrendo os pais até a raiz.

```
def reconstruct_path(self, goal_node):
    path = []
    current = goal_node
    while current is not None:
        path.append((current.x, current.y))
        current = current.parent
    return path[::-1]
```

G. Navegação do Robô

O robô segue o caminho por meio de um controle proporcional simples que ajusta a velocidade linear e angular para se alinhar ao próximo waypoint. Trecho extraído do `main`:

```
while target_idx < len(path):
    pose = HAL.getPose3d()
    robot_x, robot_y, robot_yaw = pose.x, pose.y, pose.yaw
    wp_px, wp_py = path[target_idx]
    wp_world = WebGUI.mapToWorld(url, wp_px, wp_py) # converte para coordenadas do mundo
    dx = wp_world[0] - robot_x
    dy = wp_world[1] - robot_y
    dist = math.hypot(dx, dy)
    if dist < 0.3:
        target_idx += 1
        continue
    desired = math.atan2(dy, dx)
    error = (desired - robot_yaw + math.pi) % (2 * math.pi) - math.pi
    angular = max(-2.0, min(2.0, 2.0 * error))
    HAL.setV(0.8)
    HAL.setW(angular)
    time.sleep(0.02)
```

H. Visualização da Árvore e do Caminho

A função `draw_tree_and_path_on_image` desenha a árvore em verde e o caminho em vermelho sobre a imagem do mapa, gerando a Figura 1.

```
def draw_tree_and_path_on_image(map_img, tree, path, color_tree=(0,255,0)):
    img = map_img.copy()
    for node in tree:
        if node.parent is not None:
            cv2.line(img, (node.x, node.y), (node.parent.x, node.parent.y), color_tree, 1)
    for i in range(len(path)-1):
        cv2.line(img, (path[i][0], path[i][1]), (path[i+1][0], path[i+1][1]), (0,0,255), 2)
    return img
```

V. RESULTADOS

Os testes foram realizados em um ambiente com obstáculos retangulares, com o robô iniciando na posição $(50, 50)$ e objetivo em $(950, 580)$ no mapa de ocupação (dimensões 600×1000 pixels). A Figura 1 mostra a árvore gerada pelo RRT (verde) e o caminho encontrado (vermelho).

Observa-se que a árvore explora eficientemente o espaço, crescendo em direção ao objetivo. O caminho encontrado é viável e composto por **73 pontos**, o que permite uma navegação suave. Embora não seja o caminho mais curto

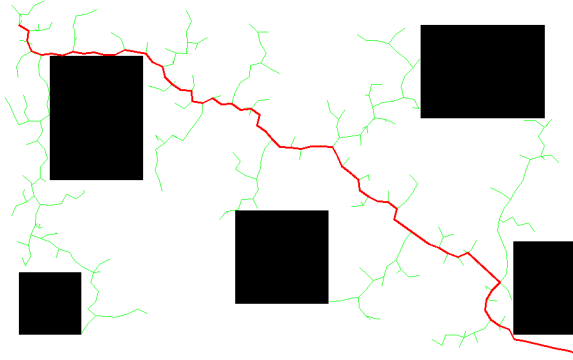


Fig. 1. Árvore gerada pelo RRT (verde) e caminho encontrado (vermelho).

possível, o que é esperado para a versão básica do RRT, ele é perfeitamente executável pelo robô.

A Figura 2 mostra o robô durante a execução da trajetória planejada, demonstrando que o controlador é capaz de seguir o caminho gerado.



Fig. 2. Robô durante a execução da trajetória planejada pelo algoritmo RRT no ambiente de simulação Gazebo.

A Tabela I apresenta as principais métricas obtidas na execução.

TABLE I
MÉTRICAS DE DESEMPENHO DO RRT

Métrica	Valor
Nós da árvore	187
Pontos no caminho	73
Sucesso na navegação	Sim

O algoritmo convergiu dentro do limite de 5000 iterações, encontrando um caminho válido. A navegação foi concluída com sucesso, com o robô atingindo o destino sem colisões.

VI. CONCLUSÃO

Este trabalho apresentou a implementação do algoritmo RRT para planejamento de caminho em robótica móvel, integrado ao ambiente RoboticsAcademy. A solução desenvolvida foi capaz de gerar árvores de exploração e encontrar caminhos viáveis em ambientes com obstáculos estáticos, além de executar a navegação do robô.

Os resultados demonstraram a eficácia do RRT em explorar o espaço de configurações e encontrar soluções rapidamente, com sucesso no cenário testado. O caminho gerado com 73 pontos mostrou-se adequado para navegação, e o controle proporcional simples foi suficiente para seguir a trajetória.

Como trabalhos futuros, propõe-se:

- Implementar variações do RRT, como RRT-Connect e RRT*, para melhorar a qualidade dos caminhos;
- Adicionar obstáculos dinâmicos para testar a capacidade de replanejamento em tempo real;
- Avaliar o desempenho do algoritmo em espaços de maior dimensionalidade (ex.: robôs manipuladores);
- Integrar técnicas de aprendizado por reforço para otimização dos parâmetros do RRT.

Dessa forma, os objetivos propostos para o desafio foram alcançados, incluindo a implementação do algoritmo, a visualização da árvore de exploração, a geração do caminho e a navegação do robô até o destino.

AGRADECIMENTOS

A autora agradece à Universidade Federal de Alagoas (UFAL), ao Projeto Novo Ensino Suplementar (NES) e ao professor Leonardo Viana pela proposta do desafio e pelo suporte durante a disciplina de Robótica Móvel. Agradece também ao JdeRobot pelo desenvolvimento da plataforma RoboticsAcademy, que tornou possível a realização prática deste trabalho.

REFERÊNCIAS

REFERENCES

- [1] S. M. LaValle, "Rapidly-exploring Random Trees: A New Tool for Path Planning," Computer Science Department, Iowa State University, Tech. Rep., 1998.
- [2] JdeRobot, "RoboticsAcademy," Disponível em: <https://jderobot.github.io/RoboticsAcademy/>, Acesso em: maio 2025.
- [3] Python Software Foundation, "Python 3 Documentation," Disponível em: <https://docs.python.org/3/>, Acesso em: maio 2025.
- [4] J. J. Kuffner Jr. and S. M. LaValle, "RRT-Connect: An Efficient Approach to Single-Query Path Planning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2000, pp. 995–1001.
- [5] S. Karaman and E. Frazzoli, "Sampling-based Algorithms for Optimal Motion Planning," in *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [6] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT*: Optimal Sampling-based Path Planning Focused via Direct Sampling of an Admissible Ellipsoidal Heuristic," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 2997–3004.
- [7] H. Choset et al., *Principles of Robot Motion: Theory, Algorithms, and Implementation*, MIT Press, 2005.